

Qiyas Data Science & AI Hackathon

Addis Ride Demand Forecasting Challenge —Instructions

Full-day hackathon · Qiyas / IADE AI Training Program

Addis Ababa University

Time-series regression · three tables to integrate · visual analysis · deployed demo

1. The Challenge

A ride-hailing company operates in 12 zones of Addis Ababa. Operations needs to know, a day or two ahead, how many trips riders will request in each zone for every hour — so it can tell drivers where to go and when. Your job: forecast the number of trips (trips) for every zone and every hour of the next 14 days, 1–14 November 2025, using the history from 1 January to 31 October 2025.

Trip history alone is not enough. You are also given an hourly weather table and a calendar of city events (holidays, football matches, concerts, conferences, road closures...). Part of the challenge is combining the three tables correctly. The weather and events tables hold real signal that a model looking only at the trip history can't see.

All three tables are raw exports — real-world messy, not cleaned for you. Expect inconsistent spelling, dates written in more than one format, missing and impossible values, duplicate rows, gaps, and more. Cleaning and joining them correctly is part of the task, and it is graded. Pay particular attention to time: a join on timestamps is only as good as the clocks behind those timestamps.

This is a full data-science workflow, not just a modeling contest. You will integrate the data, analyze it, visualize it, build and rigorously evaluate a forecasting model, and ship it as a working demo. Seven deliverables (A–G, Section 4) plus your prediction file make up your grade.

Key idea

This is a forecasting task: you predict the future from the past. That changes three habits. (1) Validate by time — train on earlier weeks, test on later ones — never with a random split. (2) Only use features that you would really know at forecast time. (3) A model that has never seen a growing trend cannot be expected to extrapolate it, so look at how demand changes over the year.

All data in this exercise is synthetic and illustrative. It was generated for training purposes and does not describe any real company; holiday dates are approximate.

2. The Data (three tables)

2.1 Trip history (the core table)

File	Rows	Contents
ride_demand_train.csv	≈85,000	Hourly trips per zone, 1 Jan – 31 Oct 2025, plus three operational columns. Explore, clean, and train on this.
ride_demand_test.csv	4,032	12 zones × 336 hours (1–14 Nov 2025). Zone and hour only — no trips column. These are the rows you predict.
submission_template.csv	4,032	row_id + predicted_trips, row_ids pre-filled in order. Fill in and submit this.

Data dictionary — trip history

Column	Type	Description
record_id / row_id	string	Unique row ID (train: record_id; test and submission: row_id). Join key for scoring.
zone	categorical	Pickup zone (12 zones). Spelled inconsistently in the raw files.
pickup_hour	text	Start of the hour, Addis Ababa local time (EAT). Written in more than one format.
trips	integer (target)	TRAIN ONLY. Trips requested in that zone in that hour. What you are predicting.
avg_fare_birr	numeric	TRAIN ONLY. Average fare in birr for that zone-hour.
avg_wait_min	numeric	TRAIN ONLY. Average rider wait in minutes.
active_drivers	integer	TRAIN ONLY. Drivers active in that zone-hour.

Not everything in the train file exists at forecast time

avg_fare_birr, avg_wait_min and active_drivers only exist for the history. In the real world you would not know the average wait or the number of active drivers for next Tuesday at 18:00 — they are consequences of demand, not causes of it. Decide for yourself whether they can legitimately be model inputs and justify it (Deliverable D4). They are still useful for analysis and for the demo.

2.2 Hourly weather — weather_hourly.csv ([join this in](#))

One row per hour for the whole city, about 7,500 rows — not merged in for you. It covers the history and the forecast fortnight. Rows for 1–14 November are weather forecasts, flagged in the data_type column, and are noisier than the observed history.

Column	Type	Description
timestamp	text	Hour of the reading. Check carefully what clock it is on — and whether the format is the same on every row.
temp_c	numeric	Air temperature.
rain_mm	numeric	Rain in the previous hour, mm.
humidity_pct	numeric	Relative humidity, %.
wind_kmh	numeric	Wind speed, km/h.
data_type	categorical	observed (history) or forecast (1–14 Nov).

This is a real join, not a lookup

A timestamp is only a number until you know which clock it uses. Before you join anything, prove — with evidence from the data, not assumption — that the weather hours and the trip hours are on the

same clock (Deliverable A2 and task B2.1). Expect gaps, duplicate hours with conflicting values, sentinel codes for "no reading", and values that look like they use a different unit for part of the year.

2.3 City events — `events_calendar.csv` (join this in)

About 165 rows: one row per event or event-like period, with a start and an end. Unlike the weather table this is not one row per hour — you must work out which trip hours fall inside, just before, or just after each event.

Column	Type	Description
event_id / event_name	string	Event identifier and name.
event_type	categorical	public_holiday, school_break, football_match, concert, conference, exhibition, road_closure, sports_run — spelled inconsistently.
venue	text	Where it happens (may be blank).
zone	text	Affected zone. May be a different spelling from the trip table, may carry extra text, may name several zones or the whole city.
start_datetime / end_datetime	text	Start and end, local time, in more than one format. Some ends are missing or earlier than the start.
expected_attendance	text	Expected crowd size, entered as free text. Often blank.
status	categorical	Whether the event actually went ahead.

Events are intervals, not timestamps

Think about how an event reaches into the hours around it: demand can rise before an event starts, during it, and especially after it ends. Which kinds of events matter, and in which zone, is for you to discover — not every row in this table affects demand, and not every demand spike is in the table.

3. Rules

1. Teams of 3–4, assigned at the start of the day.
2. Any library, any model type — baselines, linear models, trees and gradient boosting, neural nets, ensembles.
3. AI coding assistants are allowed and encouraged. This hackathon tests judgment and iteration, not typing speed.
4. Don't reverse-engineer the true trips from the test file, and don't share code or predictions across teams.

5. Your final model must use at least one feature derived from the weather table and at least one derived from the events table. A model using only calendar and zone features does not satisfy this rule.
6. Use only information that would be available at forecast time (Deliverable D4). Anything that exists only in the train file cannot be a model input.
7. Validate by time: all reported scores come from chronological splits of the train file (e.g. train before 18 Oct, validate 18–31 Oct). Random-split scores may be shown only as a contrast. Never score on the test file.
8. Learn everything from the train file only: medians, outlier caps, category lists, scalers and encoders are fitted on train and then applied to test. Never fit anything on the test file.
9. Every number, table and figure you report must be produced by code in your project folder — no hand-edited CSVs, no screenshots of spreadsheets.
10. You submit once, at the end of the day: one project folder in the layout shown in Section 6, containing your submission file and all seven deliverables.
11. All seven deliverables (Section 4) are required. Skipped items score zero, regardless of your prediction score.

Suggested team roles

With 3–4 people and one day, split the work: a data lead (A), an analysis and visualization lead (B and C), a modeling lead (D), and a deployment lead (E, plus the folder and README in G). Everyone reviews the slides (F). The pipeline from A feeds everything else, so agree early on the exact column names of your master tables and on the clock (time zone) all your tables will use.

3.1 Guide to languages and libraries

Use Python 3.10 or newer, working in Jupyter notebooks (or VS Code / Colab) plus a few .py files for the demo app. Python is what the examples and the judges' tooling assume. Teams that are strong in R may use it for A–D, but the demo (E) must then be built with a framework your team can host or run on one command. Install only what you need, and pin versions in requirements.txt.

Job	Recommended	Notes
Tables & cleaning	pandas, numpy	read_csv, to_datetime (use format= and errors=), str methods for labels, merge / merge_asof, groupby, resample. Polars is fine if the team already knows it.
Dates & time zones	pandas (tz_localize / tz_convert), zoneinfo	Africa/Addis_Ababa is UTC+3 with no daylight saving. State the clock of every table explicitly.
Static plots	matplotlib, seaborn	Required figures are saved as PNG files. seaborn heatmaps and line plots save a lot of time.
Interactive plots	plotly	Good for the stretch goal and for the demo. Export static PNGs (kaleido) for the required figures.
Classical models	scikit-learn	LinearRegression, Ridge, RandomForestRegressor, HistGradientBoostingRegressor, TimeSeriesSplit, permutation_importance, pipelines.

Job	Recommended	Notes
Gradient boosting	LightGBM, XGBoost, or CatBoost	Usually strongest on tabular data. Native handling of categorical columns helps with zones.
Time-series helpers	statsmodels (optional)	Seasonal decomposition, simple benchmarks. Prophet / neural forecasters are allowed but not required — a well-featured boosted model is a strong approach here.
Neural networks	PyTorch or Keras	Optional. Try one only after your baselines and a tree model are in place.
Tuning	Optuna, or sklearn RandomizedSearchCV	Always tune with time-ordered folds, never shuffled ones.
Saving the model	joblib	Save the fitted model (and its feature list) so the demo loads it instead of retraining.
Demo app	Streamlit (fastest) or Gradio; Flask/FastAPI with a simple HTML form	Host on Streamlit Community Cloud or Hugging Face Spaces, or run locally.
Quality checks	assert statements, optionally pandera or pytest	Used for the integrity checks in A7.
Reproducibility	requirements.txt, fixed random seeds	random_state=42 everywhere; relative paths only.

Practical advice

Start simple and add one thing at a time. A hand-built seasonal baseline plus a boosted-tree model with a handful of well-chosen features will beat a complicated model built on poorly joined data. Let an AI assistant write boilerplate (plots, app scaffolding), but check every join and every time conversion yourself.

4. Deliverables (all seven required)

Here is the full list with the weight of each part of your grade. Full scoring criteria are in the separate Judging Rubric.

Part	What you produce	Points
Prediction score	Your completed submission file, scored against the hidden answer key.	20
A — Data Cleaning & Integration Pipeline	Cleaned, time-aligned, joined, feature-engineered master tables; cleaning log; join map and audit; integrity checks.	14
B — Data Analysis Report	14 numbered analysis tasks across demand patterns, weather, events, and data quality.	14
C — Visualization Pack	12 required figures, each with a one-line takeaway.	14
D — Modeling & Evaluation	Baselines, model comparison, rolling-origin validation, leakage audit, ablation, tuning, error analysis.	14

Part	What you produce	Points
E — Deployed Forecast Demo	A working app that looks up weather and events for you and returns an hourly forecast, drivers needed and fare revenue.	8
F — 5-Slide Presentation	5 minutes + 2 minutes Q&A, with a live demo.	6
G — Project Structure & Reproducibility	Folder layout, README, requirements file, valid submission.	5
Stretch goal (optional)	Pick one of four.	5
TOTAL		100

Grading works the same way for every item: a specific number, table or chart plus a one- or two-sentence interpretation earns full credit. A result with no interpretation earns half. A correctly-reasoned "we found nothing here, and here's how we know" earns full credit — you don't have to find something dramatic to be right. A skipped item earns zero.

A — Data Cleaning & Integration Pipeline (14 pts)

Turn three raw exports into one trustworthy modeling table — and prove it is trustworthy. Everything here must be done in code, so exactly the same pipeline can be run on the test rows.

- A1 Cleaning log (2 pts): one table, one row per issue — file, column(s), issue type, how many rows affected (count and %), the fix you applied, and why. Cover all three tables: at least 5 distinct issues in the trip file, 4 in the weather file, and 4 in the events file.
- A2 Time & key standardization (2 pts, incl. the time-zone proof): (a) for zone (in all three tables) and event_type, show the unique values before and after cleaning, so every table uses the same 12 zone labels; (b) show how you parsed every timestamp format in each table and that nothing was misread (for example day-first vs month-first dates); (c) a short proof, using evidence from the data, of what clock each table's timestamps are on, and what conversion you applied so that all tables share one clock.
- A3 Join map & diagram (2 pts): a diagram of how the three tables connect — keys, join type, cardinality (many-to-one for weather; an interval join for events) — your rule for how an event window extends before and after the event, and why you chose each table as the "left" table.
- A4 Join audit (2 pts): for each join report the match rate, the row count before and after (it must not change for a many-to-one join — if it does, find and fix the duplicate keys), how many zone-hours had no weather and why, and what you did about them. For events, report how many events matched at least one zone-hour, how many were excluded, and why.
- A5 Join proof (1 pt): pick 3 specific zone-hours and show your work — for one affected by rain, one inside an event window, and one on a public holiday: which weather row and which event rows were attached, and what feature values they produced.
- A6 Feature engineering table (2 pts): at least 8 engineered features, with at least 3 calendar features (hour, day of week, holiday, payday...), at least 2 from weather (e.g. rain in the last 3 hours, rain class), at least 3 from events (e.g. hours until/since a match, in-window flags, attendance), and at least 1 lag/rolling or trend feature. For each: name, formula, source columns, why you expect it to help — and a column "known at forecast time? (yes/no)".

- A7 Integrity checks (1 pt): at least 6 automated checks written as code (assert statements or a `validate()` function) that print PASS/FAIL — for example: one row per zone-hour, no duplicate keys, all timestamps on the expected clock and range, only the 12 zone labels, no negative or sentinel values left, row count unchanged by the joins, train and test have the same feature columns, no feature uses information unavailable at forecast time. Include the printed output in your report.
- A8 Master tables & data dictionary (2 pts): export `master_train.csv` and `master_test.csv` (cleaned + joined + engineered, same feature columns, test without the target), plus `data_dictionary_master.csv` listing every column with its type, source, description, and how it was derived. The test rows must go through the same pipeline as train, with statistics learned from train only (Rule 8).

B — Data Analysis Report (14 pts)

One document (a notebook exported to HTML/PDF, or Markdown/Word/PDF) with each task under its own heading showing its ID. Answer every task with an actual number, table, or chart, plus one or two sentences of interpretation. Use your cleaned, joined data from Deliverable A. 1 point each.

B1 — Demand patterns

- B1.1 Volume by zone: total trips and each zone's share of the city total; mean trips per zone-hour. Which zones carry the most demand? Beware of any zone that did not operate for the whole period.
- B1.2 Hour-of-day profile by zone type: group the zones by the shape of their weekday hourly profile (for example residential, business district, transport hub, market, nightlife/airport). At which hours does each group peak, and when is it quietest?
- B1.3 Weekday vs weekend: compute the weekend-to-weekday ratio of mean trips for every zone. Which zones are busier on weekends and which collapse? Give the day-of-week shape for one zone that stands out.
- B1.4 Trend: how does city-wide demand change from January to October? Quantify it (weekly totals, growth in percent, trips per week) and say what that implies for forecasting November.

B2 — Weather

- B2.1 Timezone check: demonstrate with data what clock the weather timestamps use (for example the hour at which temperature peaks, or the alignment of rain with demand changes) and quantify what happens to the rain–demand relationship if the clocks are mis-aligned by 0 to 5 hours.
- B2.2 Rain effect by zone type: compare demand in rainy hours with similar dry hours (same zone, weekday, hour), separately for each zone type. Does rain increase demand everywhere?
- B2.3 Rain dose-response: bin rain into at least 4 classes (none, light, moderate, heavy) and show the demand ratio per class and zone type. Is the response proportional? Where does it saturate?

B3 — Events & calendar

- B3.1 Public holidays: for each holiday compute city-wide daily trips relative to the same weekday in nearby non-holiday weeks. Rank them. Do all holidays reduce demand? Which zones react locally?
- B3.2 Event-window study (football): for confirmed football matches, compare demand in the match zone in the 2 hours before the start, during the match, and in the 2 hours after the end, against the same zone-hours without events. Which window shows the largest uplift?

- B3.3 Event type ranking: for each event type, estimate the effect size on demand in its affected zone(s) in the relevant window. Rank them. Which types have no measurable effect?
- B3.4 Cancelled and unlisted events: (a) check whether events with status cancelled leave any demand footprint; (b) find at least 3 large demand spikes in the history that are not explained by any listed event, and say what they might be.

B4 — Operations & data quality

- B4.1 Operational variables vs demand: correlate trips with active_drivers, avg_wait_min and avg_fare_birr. Explain what each correlation does and does not mean, and why these columns cannot be used as forecast inputs.
- B4.2 Gaps and outages: list every period where the history has no rows. Separate genuine outages (the platform was down) from a zone that had not launched yet, and from random missing records. State how you treated each.
- B4.3 Pay-period effect: is demand higher around payday (the last days and first days of the month)? Compare against equally-detrended ordinary days and say whether the effect is large enough to keep as a feature.

C — Visualization Pack (14 pts)

Produce the 12 figures below, one file per figure, saved as PNG (at least 1200 pixels wide, or 150 dpi) in your figures/ folder under the exact file names shown. Use any plotting library, but every figure must be generated by code in your project. 1 point per figure, plus 2 points for overall figure quality.

File name	What it must show
fig01_gaps_and_missingness.png	Missing or invalid values per column for all three raw tables, and a timeline of missing hours per zone that makes the outage and the late-launching zone visible.
fig02_before_after_cleaning.png	Before-and-after distributions for at least 2 variables where cleaning visibly changed the picture (e.g. temperature, rain, trips spikes).
fig03_demand_trend_with_holidays.png	Daily (or weekly) city-wide trips January–October with the trend visible and public holidays marked.
fig04_hour_by_weekday_heatmap.png	Heatmap of mean trips by hour of day × day of week (city-wide, plus at least one zone to contrast).
fig05_zone_profiles.png	Weekday and weekend hourly profile for each zone type (small multiples).
fig06_weather_timezone_check.png	Evidence for the weather clock: for example the average daily temperature curve on the original vs. corrected clock, or rain–demand correlation against hour shift.
fig07_rain_effect.png	Demand ratio versus rain class, one line or bar group per zone type.
fig08_event_study.png	Average demand around event start (e.g. –6h to +6h) for at least 2 event types versus a non-event baseline.
fig09_holiday_effects.png	Index of daily trips for each public holiday relative to normal, sorted.
fig10_model_comparison.png	Validation RMSE of every model you tried (bars), with both baselines marked and rolling-origin error bars on at least your final model.

File name	What it must show
fig11_forecast_vs_actual.png	Predicted vs. actual hourly trips for at least 3 zones over a full validation fortnight.
fig12_feature_importance.png	Importance of your top 10+ features (permutation importance recommended), with weather and event features visually highlighted.

Figure quality (applies to every figure)

- Every figure has a title, labeled axes with units, a legend whenever there is more than one series, and fonts readable at slide size.
- One consistent visual style across the pack, and a colorblind-safe palette.
- Honest axes: bar charts start at zero unless clearly noted, and no chart is cropped or scaled to exaggerate a difference.
- A figures/figure_captions.md file with a one- or two-sentence caption per figure: what it shows, and the takeaway ("so what").

D — Modeling & Evaluation (14 pts)

Report results in one document (notebook plus a short written summary is fine). Every score is computed on held-out later weeks of the train file — never the test file. State your split dates.

- D1 Baselines (2 pts): a mean-predictor baseline and a seasonal-naive baseline (average trips for the same zone, day of week and hour in the training period, or in its most recent weeks), each with RMSE and MAE on a chronological validation fortnight.
- D2 Model comparison (2 pts): at least 3 model families beyond the baselines (e.g. ridge/linear, random forest, gradient boosting, a neural network), compared in one table on the same split and features, with RMSE, MAE and training time. Name your winner and explain why.
- D3 Rolling-origin validation (2 pts): at least 4 folds, each training on everything before a cut date and validating on the next 14 days, for your final model and your seasonal-naive baseline. Report mean $\pm$ standard deviation of RMSE and say what the spread tells you. Explain any fold where a model does unusually badly.
- D4 Feature availability & leakage audit (1 pt): a table of every candidate feature saying whether it is known at forecast time. State which columns you excluded and why. If you tried a leaky feature, show the inflated validation score and explain why it would fail in production.
- D5 Ablation (2 pts): train your final model type on identical splits with (i) calendar + zone + trend only, (ii) + weather, (iii) + events, (iv) + both, and report RMSE and MAE for each and the differences. Was each join worth the effort, and by how much?
- D6 Tuning (1 pt): a documented hyperparameter search (grid, random, or Optuna) using time-ordered validation — search space, number of trials, best parameters, and score before vs. after.
- D7 Error analysis (2 pts): (a) error by zone, hour of day and day type (weekday/weekend/holiday); (b) the 10 largest-error zone-hours listed, with a hypothesis for each group (including any caused by events that are not in the calendar).
- D8 Response to findings (1 pt): what you changed because of D7 and whether it helped. An honest "it didn't help" earns full credit.

- D9 Plain-language metric (1 pt): translate your RMSE and MAE into trips per zone-hour and as a % of mean demand, and into approximate drivers (use roughly 1.3 trips per active driver per hour from the history) — one short paragraph an operations manager could follow.

E — Deployed Forecast Demo (8 pts)

A small, working interface for your final model — not just a notebook cell. An operations manager who has never seen your code should be able to pick a zone and a day and get an hourly forecast.

- Inputs: the user chooses a zone and a date (any date in 1–14 November 2025). Nothing else.
- Integration: the user must NOT type in any weather or event information. The app looks up the weather forecast for that date and any events in or near that zone from your cleaned tables bundled with the app (e.g. `app/assets/`).
- Outputs: the 24-hour forecast as a table and a curve, the peak hour, the estimated number of drivers needed each hour (forecast trips ÷ about 1.3 trips per driver-hour), and expected gross fares (trips × the zone's average fare from the history) — plus a line showing what was looked up (e.g. "rain 4.2 mm at 17:00; football match at Addis Ababa Stadium 18:00–20:00").
- At least one in-app visual — e.g. the forecast curve against the zone's typical profile, with event windows shaded.
- Build it with whatever is fastest for you: Streamlit, Gradio, or a small Flask/FastAPI app. AI coding assistants scaffold these quickly.
- Host it at a URL (Hugging Face Spaces or Streamlit Community Cloud have free tiers). No hosting possible on the day? A local demo is fine — one command to run it (e.g. `streamlit run app/app.py`), demoed live from your laptop.
- Handle normal inputs without crashing, and give a friendly message for dates outside the supported range. It does not need to be right on bizarre inputs, just not broken.

F — 5-Slide Presentation (6 pts) — 5 min + 2 min Q&A

- Slide 1 — the problem and the three tables, in your own words. Slide 2 — cleaning & integration: your join map and the key numbers from A4 (including the clock problem). Slide 3 — what the data says: your two most interesting findings from B/C, with figures. Slide 4 — modeling & evaluation: baselines, ablation, rolling-origin results. Slide 5 — error analysis, your final score, demo link, and what you would try next.
- Use at least 2 figures from your Visualization Pack on the slides.
- Be ready to open your deployed demo live and forecast a zone and date of the judges' choice during Q&A.

G — Project Structure & Reproducibility (5 pts)

- G1 Folder structure: your project follows the layout in Section 6, with deliverables in the expected places and files named as shown.
- G2 README.md: team name and members, a one-paragraph summary, exact setup and run commands, the order to run the notebooks, where each deliverable lives, your final validation score, and your demo link (or a note that it runs locally).

- G3 Reproducibility: a requirements.txt (or environment.yml) with versions, relative file paths only, fixed random seeds, and notebooks that run top to bottom in a fresh environment.
- G4 Valid submission file: correct name, exactly the columns row_id and predicted_trips, all 4,032 row_ids once each, in the original order, no blank, negative or non-numeric predictions.
- G5 Method hygiene: the final model uses weather and event features (Rule 5), only forecast-time features (Rule 6), validation was chronological (Rule 7), and nothing was fitted on the test file (Rule 8).

5. Stretch Goal (optional, pick one — 5 pts)

- Interactive explorer: a dashboard (Plotly Dash, Streamlit, or interactive Plotly HTML) with filters for zone, date range and hour that updates at least 3 charts.
- Zone equity: does your forecast do worse in any zone — for example the newest one or a nightlife zone? What would under-forecasting mean for drivers and riders there? Propose one mitigation.
- Uncertainty: produce a prediction interval per zone-hour (quantile regression or residual-based) and explain how an operations manager should use it when deploying drivers.
- Monitoring & drift memo: one page — how would you know if this forecast degrades over time (growth, new zones, unplanned events, weather changes)? What would you monitor, and when would you retrain?

6. Submission & Folder Structure

You submit once, at the end of the day, a Git repository link. Use exactly this layout so judges can find everything in easier.

6.1 Folder layout

```
team_<NAME>/
+-- README.md                setup, run order, summary, demo link    (G2)
+-- requirements.txt          pinned package versions              (G3)
|
+-- submission/
|   +-- team_<NAME>_submission.csv    the file that gets scored      (G4)
|
+-- data/
|   +-- raw/                        the original 5 CSVs, never edited
|   +-- processed/
|       +-- master_train.csv          (A8)
|       +-- master_test.csv           (A8)
|       +-- data_dictionary_master.csv (A8)
|
+-- notebooks/                  (or one notebook with clear A-D headings)
|   +-- 01_cleaning_and_integration.ipynb (A)
|   +-- 02_analysis_report.ipynb        (B)
|   +-- 03_visualizations.ipynb         (C)
|   +-- 04_modeling_and_evaluation.ipynb (D)
|
+-- src/                        optional: reusable pipeline code
|   +-- cleaning.py  features.py  train.py  predict.py
|
+-- models/
|   +-- final_model.joblib            the trained model the demo loads
```

```

+-- figures/
|   +-- fig01_gaps_and_missingness.png ... fig12_feature_importance.png      (C)
|   +-- figure_captions.md                                                  (C)
|
+-- reports/
|   +-- A_cleaning_and_integration.(pdf|md|html)    log, join map, audit, proof (A)
|   +-- B_analysis_report.(pdf|md|html)            (B)
|   +-- D_model_evaluation.(pdf|md|html)           (D)
|
+-- app/                                          (E)
|   +-- app.py                                  Streamlit / Gradio / Flask entry point
|   +-- requirements.txt
|   +-- assets/                                cleaned weather + events tables, model
|
+-- presentation/
|   +-- team_<NAME>_slides.(pdf|pptx)          (F)

```

6.2 Where each deliverable goes

Deliverable	What judges will look for	Location
Prediction score	The scored file	submission/team_<NAME>_submission.csv
A — Pipeline	Code, written log/map/audit/proof/feature table/check output, and the three exported data files	notebooks/01_..., reports/A_..., data/processed/
B — Analysis	All 14 tasks, each under its ID heading	reports/B_analysis_report.*, notebooks/02_...
C — Visualizations	12 PNGs with the exact names, plus captions	figures/, notebooks/03_...
D — Modeling	D1–D9 write-up, code that reproduces it, the saved final model	reports/D_..., notebooks/04_..., models/
E — Demo	App code, bundled lookup tables, and the demo URL	app/, URL in README.md
F — Slides	Your 5 slides	presentation/
G — Structure	README, requirements, folder layout	project root

6.3 Naming and hygiene rules

- Lowercase file and folder names with underscores — no spaces, no special characters. Number your notebooks (01_, 02_ ...) in run order.
- Relative paths only: code must never contain a path like C:\Users\... or /content/drive/... Anything it reads should live inside the project folder.
- Never edit files in data/raw/. All cleaning happens in code that writes to data/processed/.
- Reports can be PDF, HTML, Markdown, or Word. A notebook exported to HTML counts, as long as each task or figure sits under its ID heading (A1, B2.3, D5, ...).
- One notebook instead of four is fine if its sections are clearly headed with the deliverable IDs — but the folders, figures, README, and submission file stay as shown.

- Leave out virtual environments (venv/, .venv/), __pycache__/, .ipynb_checkpoints/, and any file over 100 MB.

6.4 Pre-submission checklist

- Submission CSV opens, has 4,032 rows, no blank or negative predictions, no duplicate row_ids.
- All 12 figures exist under the exact names, and figure_captions.md has 12 entries.
- master_train.csv, master_test.csv and data_dictionary_master.csv exist in data/processed/.
- Notebooks run top to bottom from a fresh environment with the commands in your README.
- Your demo URL works in a private/incognito window — or the one-line local run command is in the README.
- Slides are in presentation/ and you have rehearsed the live demo.

7. FAQ

Can we drop a column we think isn't useful?

Yes — that's a legitimate, gradeable decision, as long as you investigated it first and say so in your report.

A row looks impossible — can we drop or cap it?

Yes. State what you did and why in your cleaning log (A1). Remember that you must still forecast every zone-hour in the test file, so think about how your pipeline handles bad values rather than just deleting rows.

Does the weather table cover the days we have to forecast?

Yes, with forecast values (data_type = forecast). They are less accurate than observed values, as in real life. Your model's real-world performance depends on the forecast, not on perfect weather.

Some events in the calendar didn't happen — what do we do?

Decide, using the data, how to treat them. Your decision and evidence go in A4 and B3.4.

Do we really need all 12 figures and all 14 analysis tasks if time is short?

Yes — every item is graded and a skipped item scores zero. That's why the work is built to be split across a team. Quick, correct and clearly interpreted beats elaborate but unfinished.

We can't get public hosting working — what now?

Run it locally and demo it live from your own machine during your presentation. Say so in your README and on your final slide. You won't be penalized for a hosting issue outside your control, only for not having a working demo at all.

How exactly is our prediction score calculated?

It is based on the RMSE of your submission against the hidden answer key, scaled between a "weak" floor and a strong-reference ceiling. See the Judging Rubric for the formula.